



APOLLO INSTITUTE OF COMPUTER APPLICATION

Bachelor of Computer Applications

Semester 2

Data Structure (BC02001011) Assignment 5

Issue Date: 08-04-2026

Submission Date:

UNIT 5: SORTING AND SEARCHING

Q.1. Explain the working of Sequential Search algorithm with suitable examples

◇ **Definition**

Sequential Search (also known as Linear Search) is a searching technique in which each element of the list is checked one by one until the required element is found or the list ends.

◇ **Algorithm (Steps)**

1. Start from the first element of the list
2. Compare the current element with the search key
3. If both are equal → element is found
4. If not equal → move to the next element
5. Repeat steps until the element is found or the list ends
6. If element is not found after checking all elements → return “Not Found”

◇ **Flow of Working (Concept)**

☞ It follows a **linear approach**:

First element → Second → Third → ... → Last element

◇ **Example (Element Found Case)**

Array: 10, 20, 30, 40, 50

Search Key = 30

Step	Index	Element	Comparison	Result
1	0	10	$10 \neq 30$	Continue
2	1	20	$20 \neq 30$	Continue
3	2	30	$30 = 30$	Found

👉 Element found at **position 3**

◇ Example (Element Not Found Case)

Array: 10, 20, 30, 40, 50

Search Key = 60

Step	Element	Result
1	10	Not Found
2	20	Not Found
3	30	Not Found
4	40	Not Found
5	50	Not Found

👉 Final Result: **Element Not Found**

◇ Time Complexity

- Best Case: **O(1)** (element found at first position)
- Worst Case: **O(n)** (element at last or not present)

◇ Advantages

- Simple and easy to understand
- Works on **unsorted data**
- No extra memory required

◇ Disadvantages

- Slow for large datasets
- Not efficient compared to Binary Search

◇ Conclusion

Sequential Search is a basic searching technique suitable for small or unsorted datasets. However, it becomes inefficient for large data due to its linear time complexity.

Q.2. Explain the working of Binary Search algorithm with suitable examples

◇ Definition

Binary Search is an efficient searching technique used to find an element in a **sorted array** by repeatedly dividing the search space into two halves.

◇ Condition

☞ The list/array must be **sorted** (ascending or descending)

◇ Algorithm (Steps)

1. Initialize **low = 0** and **high = n - 1**
2. Calculate **mid = (low + high) / 2**
3. Compare the middle element with the search key
4. If element = key → return position (Found)
5. If key < element → search in left half (**high = mid - 1**)
6. If key > element → search in right half (**low = mid + 1**)
7. Repeat steps until **low ≤ high**
8. If not found → return “Not Found”

◇ Working Concept

☞ Binary Search reduces the search space by **half in each step**, making it very fast.

◇ Example (Element Found Case)

Array: 10, 20, 30, 40, 50

Search Key = 40

Step	Low	High	Mid	Element	Comparison	Result
1	0	4	2	30	40 > 30	Search Right
2	3	4	3	40	40 = 40	Found

☞ Element found at **position 4**

◇ Example (Element Not Found Case)

Array: 10, 20, 30, 40, 50

Search Key = 35

Step	Low	High	Mid	Element	Result
1	0	4	2	30	Right
2	3	4	3	40	Left
3	3	2	—	—	Stop

👉 Final Result: **Element Not Found**

◇ **Time Complexity**

- Best Case: **$O(1)$** (middle element is key)
- Worst Case: **$O(\log n)$**

◇ **Advantages**

- Very fast and efficient
- Suitable for large datasets
- Reduces comparisons significantly

◇ **Disadvantages**

- Requires **sorted data**
- Slightly complex compared to linear search

◇ **Conclusion**

Binary Search is one of the most efficient searching techniques. It works best on sorted data and significantly reduces search time compared to Sequential Search.

Q.3. Sort the following list using Bubble Sort and show all passes:

98, 37, 56, 82, 11, 77, 64, 45, 23, 99

◇ **Definition**

Bubble Sort is a sorting technique in which adjacent elements are compared and swapped if they are in the wrong order. After each pass, the largest element moves to its correct position.

◇ **Given List**

98, 37, 56, 82, 11, 77, 64, 45, 23, 99

◇ **Pass-wise Table**

Pass	Array After Pass
Pass 1	37, 56, 82, 11, 77, 64, 45, 23, 98, 99
Pass 2	37, 56, 11, 77, 64, 45, 23, 82, 98, 99
Pass 3	37, 11, 56, 64, 45, 23, 77, 82, 98, 99
Pass 4	11, 37, 56, 45, 23, 64, 77, 82, 98, 99
Pass 5	11, 37, 45, 23, 56, 64, 77, 82, 98, 99
Pass 6	11, 37, 23, 45, 56, 64, 77, 82, 98, 99
Pass 7	11, 23, 37, 45, 56, 64, 77, 82, 98, 99
Pass 8	No change
Pass 9	No change

◇ **Final Sorted List**

☞ **11, 23, 37, 45, 56, 64, 77, 82, 98, 99**

Q.4. Explain the working of Quick Sort. How is it different from Merge Sort? Illustrate Quick Sort with an example by sorting the following list:

10, 80, 30, 90, 40, 50, 70

◇ **Definition (Quick Sort)**

Quick Sort is a **divide-and-conquer sorting algorithm** that selects a **pivot element** and partitions the array into two parts such that:

- Elements smaller than pivot → left side
- Elements greater than pivot → right side

Then both parts are sorted recursively.

◇ Algorithm (Steps)

1. Select a pivot element (first/last/middle)
2. Rearrange elements so that:
 - Smaller elements come before pivot
 - Larger elements come after pivot
3. Place pivot in its correct position
4. Recursively apply Quick Sort on left and right subarrays

◇ Working with Example

◇ Given List

10, 80, 30, 90, 40, 50, 70

◇ Step 1: Choose Pivot = 10

Elements smaller than 10 → None

Elements greater than 10 → 80, 30, 90, 40, 50, 70

🔑 Array becomes:

10 | 80, 30, 90, 40, 50, 70

◇ Step 2: Apply Quick Sort on Right Part

Choose Pivot = 70

Rearrange:

30, 40, 50 | 70 | 80, 90

◇ Step 3: Sort Left Subarray (30, 40, 50)

Pivot = 30

→ 30, 40, 50 (already sorted)

◇ Step 4: Sort Right Subarray (80, 90)

Pivot = 80

→ 80, 90

◇ Final Sorted List

🔑 10, 30, 40, 50, 70, 80, 90

◇ Time Complexity

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n^2)$

◇ Advantages of Quick Sort

- Very fast in practice
- Requires less memory
- Efficient for large datasets

◇ Disadvantages of Quick Sort

- Worst-case performance is poor
- Not a stable sorting algorithm

📦 Difference Between Quick Sort and Merge Sort

Feature	Quick Sort	Merge Sort
Technique	Partition	Divide & Merge
Time Complexity	$O(n \log n)$ avg	$O(n \log n)$
Worst Case	$O(n^2)$	$O(n \log n)$
Space	In-place (less memory)	Requires extra memory
Stability	Not stable	Stable
Speed	Faster in practice	Slightly slower

◇ Conclusion

Quick Sort is faster and memory-efficient, while Merge Sort is more stable and predictable. Quick Sort is widely used due to its practical performance.

Q.5. Difference between bubble sort and selection sort

◇ Introduction

Bubble Sort and Selection Sort are simple comparison-based sorting algorithms used to arrange elements in ascending or descending order. However, their working methods are different.

◇ Difference Table

Feature	Bubble Sort	Selection Sort
Working Method	Compares and swaps adjacent elements	Selects minimum element and places it at correct position
Process	Repeated swapping of neighboring elements	Repeated selection of smallest element
Number of Swaps	More swaps	Fewer swaps
Time Complexity	$O(n^2)$ (worst & average)	$O(n^2)$ (worst & average)
Best Case	$O(n)$ (already sorted)	$O(n^2)$
Stability	Stable	Not stable
Efficiency	Less efficient	Slightly more efficient
Memory Usage	No extra memory	No extra memory
Adaptability	Adaptive (can stop early)	Not adaptive

◇ Working Explanation

◇ Bubble Sort

- Compares adjacent elements
- Swaps if elements are in wrong order
- After each pass, the largest element moves to the end

☞ Example:

5, 3, 2 $\rightarrow$ 3, 2, 5 $\rightarrow$ 2, 3, 5

◇ Selection Sort

- Finds the minimum element
- Places it at the correct position
- Repeats for remaining array

☞ Example:

5, 3, 2 $\rightarrow$ 2, 3, 5

◇ Advantages

Bubble Sort

- Easy to understand
- Stable sorting

Selection Sort

- Fewer swaps
- Better for memory-sensitive systems

◇ Disadvantages

- Both are slow for large datasets
- Not suitable for real-world large applications

◇ Conclusion

Bubble Sort is simple and stable but performs more swaps, while Selection Sort is slightly more efficient due to fewer swaps but is not stable. Both algorithms are mainly used for educational purposes.

Q.6. Explain Quick sort and Merge sort with examples.

◇ Introduction

Quick Sort and Merge Sort are efficient sorting algorithms based on the **divide-and-conquer technique**. They are widely used for sorting large datasets.

◇ 1. Quick Sort

◇ Definition

Quick Sort is a sorting algorithm that selects a **pivot element** and partitions the array into two parts such that smaller elements are on the left and larger elements are on the right.

◇ Algorithm (Steps)

1. Select a pivot element
2. Partition the array

3. Place pivot at correct position

4. Apply Quick Sort recursively on left and right subarrays

◇ **Example**

Array: 5, 3, 8, 2

Step 1: Pivot = 5

→ Left: 3, 2 | Pivot: 5 | Right: 8

Step 2: Sort left part

→ 2, 3

🔗 Final Sorted Array:

2, 3, 5, 8

◇ **Time Complexity**

- Best/Average: $O(n \log n)$
- Worst: $O(n^2)$

◇ **Advantages**

- Very fast in practice
- Requires less memory

◇ **Disadvantages**

- Worst-case performance is poor
- Not stable

◇ **2. Merge Sort**

◇ **Definition**

Merge Sort divides the array into smaller parts, sorts them, and then merges them to get the final sorted array.

◇ **Algorithm (Steps)**

1. Divide array into two halves
2. Recursively sort each half
3. Merge the sorted halves

◇ Example

Array: 38, 27, 43, 3

Step 1: Split

→ [38, 27] and [43, 3]

Step 2: Sort

→ [27, 38] and [3, 43]

Step 3: Merge

→ **3, 27, 38, 43**

◇ Time Complexity

- Best/Worst: $O(n \log n)$

◇ Advantages

- Stable sorting algorithm
- Works well for large datasets

◇ Disadvantages

- Requires extra memory
- Slightly slower than Quick Sort in practice

Difference Between Quick Sort and Merge Sort

Feature	Quick Sort	Merge Sort
Technique	Partition	Divide & Merge
Time Complexity	$O(n \log n)$ avg	$O(n \log n)$
Worst Case	$O(n^2)$	$O(n \log n)$
Space	Less	More
Stability	Not stable	Stable
Speed	Faster	Slightly slower

◇ **Conclusion**

Quick Sort is faster and memory-efficient, while Merge Sort is stable and gives consistent performance. Both are important for efficient data sorting.